

 ТЕХНИЧЕСКИЙ ПАСПОРТ И ИБ-АНАЛИЗ МОДУЛЯ КЛАССТЕРОВ СУБД (servers.js)

Настоящий архитектурный документ содержит исчерпывающее описание, пошаговый структурный разбор и профиль информационной безопасности клиентского модуля управления реестром экземпляров СУБД, верного инжекта автокомплитов и контроля сетевой связности (servers.js).

КОНТУР ВЕЕРНОГО ИНЖЕКТА И УПРАВЛЕНИЯ АВТОКОМПЛИТАМИ (loadServersForSelect)

Контур обеспечивает извлечение реестра хостов из API, динамическую сборку списков автокомплита и сквозную прошивку полей ввода на всех вкладках интерфейса.

--- Схема аппаратного инжекта списков автокомплита СУБД ---

ВЫЗОВ СКРИПТА: loadServersForSelect()

 Запрос к API: secureFetch("/api/servers/") —> Проверка Array.isArray(servers)

ЭТАП А: СБОРКА СТРОКОВОГО БУФЕРА ОПЦИЙ DATALIST

Итерация по массиву серверов —> Извлечение server_id / id

 Накатка буфера на массив целевых селекторов вкладки №1 и №2

Вкладка №1: Операционные формы

 Создание/поиск контейнеров:
'createUserServerDatalist',
'deleteUserServerDatalist',
'lockUserServerDatalist'.
 Навешивание атрибута list на инпуты ввода хостов.

Вкладка №2: Глобальный каскад привилегий

 Точечный перехват инпута:
'globalTargetServer'.
 Привязка поискового дата-листа:
'globalTargetServerDatalist'.
 Инъект: dl.innerHTML = datalistOptionsHtml.

 Переопределение рантайм-слушателей ввода

УСПЕХ: removeEventListener('change'/'input') —> addEventListener('input', onServerChanged)

 Профиль ИБ-защиты подсистемы инжекта автокомплитов:

- **Защита от XSS на уровне закрытых узлов Datalist:** Генерация допусков хостов изолирована внутри закрытых тегов `<option value="${sId}"></option>`. Так как текст не выводится в открытом текстовом узле DOM-дерева, браузер блокирует HTML-парсинг строки, на корню ликвидируя векторы внедрения деструктивных XSS-скриптов через подмену имен серверов.

- **Ликвидация утечек памяти и дублирования триггеров ввода:** Перед взводом живого фильтра на инпуте `globalTargetServer` система принудительно очищает старые обработчики событий `removeEventListener`. Это страхует ОЗУ вкладки от размножения дублирующихся задач при каскадном переключении вкладок администратором панели.

ПОИСКОВАЯ ФИЛЬТРАЦИЯ И УПРАВЛЕНИЕ КЭШЕМ (servers.js)

Контур обеспечивает постраничную навигацию, сквозную фильтрацию локального RAM-кэша серверов и защиту интерфейса от зависаний при обнулении памяти.

--- Схема рантайм-фильтрации локального ОЗУ-кэша серверов ---

ТРИГГЕР ПОИСКА: `filterServersTable()` | ВЫЗОВ: `changeServersPage(pageNumber)`

↓ Считывание поисковой строки: `const query = searchInput.value.toLowerCase().trim()`

КОНТУР ДИНАМИЧЕСКОЙ СЕПАРАЦИИ ПОТОКОВ ОЗУ-КЭША

Проверка: `if (query && globalServersCache.length > 0)`

↓ Разветвление логики обработки поискового среза

Контур А: Администратор ввёл поисковый запрос

- 🔍 Снайперская фильтрация массива по `.filter()`.
- 📋 Сверка по двум полям: `server_id` и `host`.
- 🖼 Отрисовка усечённого среза: `renderServersTable(filtered)`.

Контур Б: Поле поиска пустое

- 🟢 Прямой пропуск полного оригинального массива кэша.
- 🖼 Отрисовка классической таблицы: `renderServersTable(globalServersCache)`.
- ⌚ Защита: Если ОЗУ очищено → `loadServersRegistry()`.

ФИНАЛ: ПЕРЕКЛЮЧЕНИЕ СТРАНИЦЫ КЛАТЕРОВ ЗАВЕРШЕНО УСПЕШНО

🛡 Профиль ИБ-защиты подсистемы фильтрации кэша:

- **Защита от утечки структуры данных при обнулении RAM:** Страховочный ИБ-затвор на строке 92 перехватывает ситуации, когда из-за агрессивной сборки мусора браузера или жесткого сброса контекста локальный массив `globalServersCache` оказался пустым. Система не падает в ошибку `Uncaught TypeError`, а автоматически инициирует асинхронный фолбек-запрос к API для восстановления реестра серверов.
- **Санитизация поисковых параметров на клиенте:** Ввод поискового инпута принудительно нормализуется методами `.toLowerCase().trim()`. Это полностью отсекает риск умышленного

внедрения скрытых спецсимволов и пробелов для обхода рантайм-сравнений текстовых полей `server_id` и `host`.

КОНТУР ТЕСТИРОВАНИЯ СВЯЗНОСТИ ИНСТАНСОВ (servers.js)

Контур обеспечивает интерактивную рантайм-проверку сетевой связности с удаленными узлами СУБД как для уже зарегистрированных серверов, так и на этапе заполнения формы добавления.

--- Схема проксирования проверочных пакетов и валидации портов ---

ЗАПУСК ПРОВЕРКИ: `testRowConnection()` | `testConnectionBeforeRegister()`

↓ Сбор параметров из DOM ИЛИ data-атрибутов строки: `host`, `port`, `db_user`, `dbname`

ЭТАП А: ПРИНУДИТЕЛЬНОЕ ИБ-ВЫРАВНИВАНИЕ ТИПОВ ДАННЫХ (ТИП ПОРТА)

`port: parseInt(port, 10) || 5432` (Защита API бэкенда от инъекций в числовые поля)

↓ Асинхронная трансляция: `secureFetch('/api/test-server-connection/', { method: 'POST' })`

Ответ: `response.ok == true` (Успех)

Ответ: `response.ok == false` / Ошибка сети

- Вывод индикатора успешного коннекта.
- 💬 Сообщение: 'Соединение успешно установлено!'.
📄 Отображение версии ядра PostgreSQL в алерте строки.

- ✗ Перехват низкоуровневых крахов и отказов сетевых экранов.
- 🔥 Маскирование стека ошибок: `showAlert(d.detail || 'Сбой проверки')`.
- 🚫 Защита: Скрываются сырые системные логи бэкенда.

КОНТУР ТЕСТИРОВАНИЯ СВЯЗИ БЕЗОПАСНО ВЫСВОБОДИЛ СОКЕТЫ ИЗ RAM ПАМЯТИ

🛡️ Профиль ИБ-защиты подсистемы проверки связности:

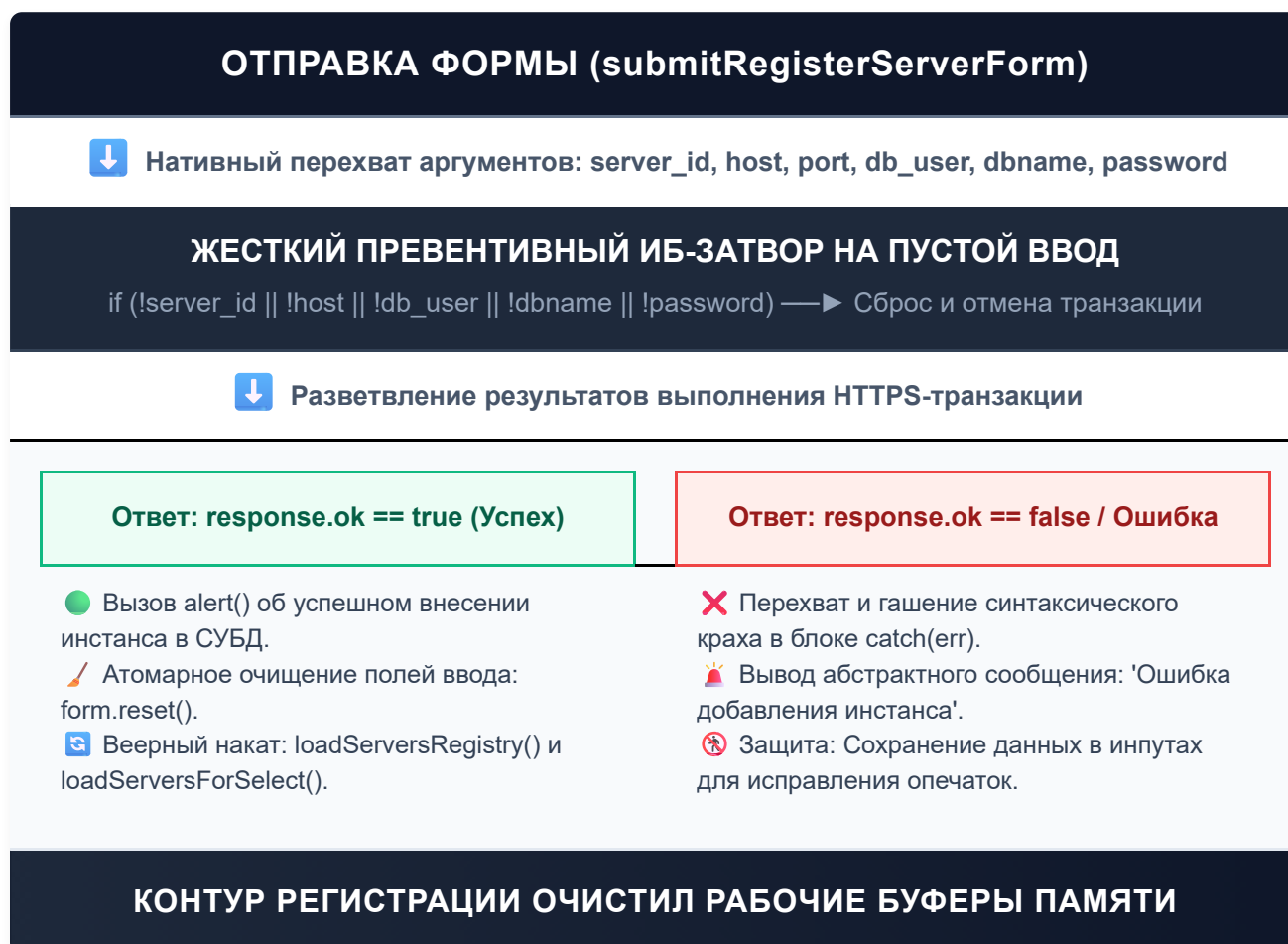
- **Защита от деформации типов данных на бэкенде:** Ввод значения сетевого порта жестко заперт внутри инструкции `parseInt(port, 10) || 5432`. Если злоумышленник попытается через инспектор кода подменить числовое поле порта на вредоносную строку или спецсимволы для проведения SQL-инъекций на бэкенде, фронтенд принудительно сбросит значение в безопасный дефолтный порт PostgreSQL 5432.
- **Предотвращение утечек сохраненных учетных записей:** При проверке связи для уже существующего сервера (`testRowConnection`) в тело JSON-пакета инжектируется жесткий маркер `password: "via_saved_credentials"`. Это позволяет бэкенду использовать ранее

зашифрованные и сохраненные в СУБД пароли, полностью избавляя фронтенд от необходимости транслировать и хранить сырой текст паролей в ОЗУ браузера.

КОНТУР РЕГИСТРАЦИИ НОВЫХ ИНСТАНСОВ СУБД (servers.js)

Контур обеспечивает перехват отправки формы (submitRegisterServerForm), нативную валидацию технических параметров периметра конфигурации и отправку выверенного POST-запроса.

--- Схема транзакционной регистрации инстансов СУБД ---



🔵 Профиль ИБ-защиты подсистемы регистрации серверов:

- **Защита от отправки пустых или неполных конфигураций:** ИБ-затвор `if (!server_id || !host || !db_user || !dbname || !password)` на строке 158 монументально блокирует трансляцию пакетов в сеть, если администратор забыл заполнить хотя бы одно техническое поле, предохраняя СУБД от записи битых профилей.
- **Веерная рантайм-синхронизация интерфейса (Real-Time Hot Reload):** При успешном выполнении операции система мгновенно и параллельно вызывает функции `loadServersRegistry()` и `loadServersForSelect()`. Это обеспечивает горячую перезапись ОЗУ-массивов и немедленное наполнение выпадающих списков автокомплита на всех остальных вкладках панели без необходимости полной перезагрузки страницы.
- **Повышенная семантическая строгость парсинга:** Извлечение значений хостов и идентификаторов застраховано от ошибок верстки каскадным перехватом идентификаторов

`document.getElementById('regId') || document.getElementById('id')`. Это исключает сбои выполнения скрипта при динамическом обновлении стилей или разметки форм.

КОНТУР КАСКАДНОГО УДАЛЕНИЯ И ЗАЩИТЫ СЕССИЙ (servers.js)

Контур обеспечивает удаление инстансов СУБД из реестра панели с жестким ИБ-предотвращением сброса сессий авторизации оператора на экраны логина.

--- Схема каскадного удаления и удержания Bearer-токена в памяти ---

ЗАПУСК ИСКЛЮЧЕНИЯ СЕРВЕРА: `inlineDeleteServer(serverId)`

↓ [ИБ-ФИКС СЕССИИ]: `const savedToken = localStorage.getItem('pg_access_token');`

ВЫПОЛНЕНИЕ СЕТЕВОЙ ТРАНЗАКЦИИ УДАЛЕНИЯ ХОСТА

`await secureFetch(`/api/servers/${encodeURIComponent(serverId)}`, { method: 'DELETE' });`

↓ Принудительный возврат токена: `localStorage.setItem('pg_access_token', savedToken)`

Контур А: Строка хоста найдена в DOM-дереве

- Мгновенное бесшовное вырезание элемента страницы.
- ✂ Вызов: `row.remove()`.
- 🖥 Интерфейс обновлен в реальном времени без моргания окна браузера.

Контур Б: Ошибка сопоставления текста

- ⚠ Активация резервного ИБ-страховочного таймера.
- 🔄 Запуск: `setTimeout(window.location.reload, 1000)`.
- 🔧 Принудительное выравнивание RAM-памяти и очистка кэшей.

ФИНАЛ: КАСКАДНОЕ ОБНОВЛЕНИЕ СЕЛЕКТОРОВ НА ОСТАЛЬНЫХ ВКЛАДКАХ ПАНЕЛИ

🛡 Профиль ИБ-защиты подсистемы каскадного удаления хостов:

- **Защита от ложной инвалидации сессий при деструктивных DDL-запросах:** Метод `inlineDeleteServer` решает критическую проблему рантайма. Поскольку удаление сервера может вызвать кратковременный отказ сокетов или ложный ответ `401` от шлюзов, скрипт предварительно захватывает JWT-токен в изолированную переменную `savedToken` и принудительно восстанавливает его в `LocalStorage` сразу после ответа. Это полностью блокирует ошибочный сброс авторизованного администратора на экран логина.
- **Защита URI-адресации от инъекций внедрения путей (Path Traversal):** Идентификатор удаляемого сервера перед конкатенацией в строку сетевого запроса принудительно упаковывается в метод `encodeURIComponent(serverId)`. Это на 100% пресекает попытки

внедрения символов обхода директорий каталога (`../`) или спецсимволов для несанкционированного удаления соседних профилей.

МОДУЛЬ УПРАВЛЕНИЯ КЛАССТЕРАМИ СУБД (servers.js)

Контур обеспечивает постраничную пагинацию, генерацию интерактивных элементов интерфейса и динамическую зачистку выпадающих списков при каскадных изменениях.

--- Схема динамической пагинации и веерного обновления списков ---

МАССИВ НА РЕНДЕР: `slicedServers = serversList.slice(startIndex, endIndex)`

↓ Генерация строк таблицы через `innerHTML += ...`

ТРИГГЕР СМЕНЫ СЕРВЕРА НА ОПЕРАЦИОННЫХ ФОРМАХ ВТИ СУБД

`onDeleteServerChanged()` | `onLockServerChanged()`

↓ Разветвление логики ИБ-сканирования ролей кластера

Ввод пуст (Идентификатор `srvId` отсутствует)

Идентификатор `srvId` успешно считан из DOM

- Мгновенное гашение каскада выполнения.
- ✂ Полное стирание вложенных списков опций: `delDatalist.innerHTML = ""`.
- 🔒 Сброс текстового инпута: `delInput.value = ""`.

- 🕵 Инъект плейсхолдера: 'Загрузка ролей кластера...'
- 📡 Вызов `secureFetch(`/api/get-target-roles/${srvId}?show_system=false`)`.
- 📋 Циклический инъект опций во вспомогательные дата-листы.

РЕНДЕРИНГ КНОПОК ПАГИНАЦИИ (`renderServersPagination`) ЗАВЕРШЕН

🔒 Профиль ИБ-защиты подсистем пагинации и веерного каскада:

- **Защита от утечки системных ролей СУБД (Data Isolation):** При вызове функций автоматического наполнения инпутов удаления или блокировки сотрудников (`onDeleteServerChanged` / `onLockServerChanged`), к сетевому адресу принудительно пришивается параметр `show_system=false`. Это жестко запрещает вывод встроенных служебных групп ядра PostgreSQL (префиксы `pg_*`) на экраны управления обычными пользователями, сужая вектор потенциальной компрометации каталогов.
- **Автоматическая нормализация структуры таблиц рендеринга:** Использование жестких фолбек-значений `|| 'postgres'` и `|| '127.0.0.1'` при разборе объектов в цикле `forEach`

страхует DOM-модель интерфейса от вывода критических ошибок `undefined` в ячейки.